

CMSC 471 PA 1

Suhaani Thakur Section 02

Q1) Notation for all three: s is the current word, g is the goal word, and $d(s)$ is the number of positions where s and g have different letters. All three heuristics return 0 when $s = g$, since then $d(s) = 0$.

1. Precise formula

- a) Steps: $h(s) = d(s)$, the number of letters that don't match the goal.
- b) Scrabble: $h(s) = \text{sum of value}(g[i])$ over every position i where $s[i] \neq g[i]$. $\text{value}()$ is the letter's Scrabble value from the assignment table.
- c) Frequency: if $d(s) = 0$, then $h(s) = 0$; otherwise

$$h(s) = (d(s) - 1) * (1 + r_{\min}) + (1 + \text{rarity}(g))$$

where $r_{\min}$ is the smallest rarity of any dictionary word with the same length as s . In `words34.txt.gz`, that's 0.016226 for 3-letter words ("the") and 0.351506 for 4-letter words ("that"). `dc.py` computes it once when it loads the dictionary.

2. What unavoidable remaining cost it lower-bounds

- a) Steps: the number of actions still needed. Every action costs 1 under steps, so that's also the remaining cost.
- b) Scrabble: the cost of the goal letters that still have to be put into the wrong positions.
- c) Frequency: the cost of the fewest actions still needed. The last action always produces the goal word, so its cost $(1 + \text{rarity}(g))$ is known exactly, and every other action is charged the cheapest cost any word could have $(1 + r_{\min})$.

3. Why it cannot overestimate the true optimal remaining cost

- a) Steps: Each action only changes one letter, so it can fix at most one wrong position. With $d(s)$ wrong letters, I need at least $d(s)$ more actions, and each one costs 1, so the remaining cost is at least $d(s)$. The real path can be longer if the in-between words aren't in the dictionary (why -> ask has $d = 3$ but takes 5 steps), but it can never be shorter. This is the same idea as the "tiles out of place" heuristic for the 8-puzzle from class.
- b) Scrabble: Every wrong position i have to get the goal letter $g[i]$ put there at some point before we reach the goal, and since you pay for the new letter, that action costs exactly $\text{value}(g[i])$. Each action only changes one position, so fixing different positions takes different actions, and their costs add up. Any extra moves (detours or changing a letter and then changing it again) only add more cost, since every letter is worth at least 1. So the real cost can't be less than the sum. For bear -> duck, $h = d(2) + u(1) + c(3) + k(5) = 11$, which is exactly the optimal cost.
- c) Frequency: Like steps, I need at least $d(s)$ more actions. The last one always lands on the goal word, so it costs exactly $1 + \text{rarity}(g)$. Every other action lands on some dictionary word of the same length, and $r_{\min}$ is the smallest rarity of any word of that length, so each costs at least $1 + r_{\min}$. So the real cost is at least $(d(s) - 1) * (1 +$

$r_{\min} + 1 + \text{rarity}(g)$, which is $h(s)$. If the path takes more than $d(s)$ steps, it only costs more, so h still doesn't overestimate.

Additional note: consistency

The A* in the supplied search.py is the graph-search version (it keeps an explored set), so I also checked that all three heuristics are consistent: $h(n)$ less than or equal to $\text{cost}(n, n') + h(n')$ for every action $n \rightarrow n'$.

- a) Steps: one action changes d by at most 1, so h drops by at most 1, which is the action cost.
- b) Scrabble: h only goes down when the new letter is the goal letter for that position, and then it drops by exactly that letter's value, which is the action cost.
- c) Frequency: if n' is the goal, then $d(n) = 1$ and $h(n) = 1 + \text{rarity}(g)$, which is exactly the action cost. Otherwise d drops by at most 1, so h drops by at most $1 + r_{\min}$ less than or equal to $1 + \text{rarity}(n')$, which is the action cost.

I did a sanity check; every delta printed by dctest.py and dcsolve.py was less than or equal to 0. That only covers the returned paths, so it doesn't prove admissibility by itself. The arguments above are the actual proof.

Q2) I ran `uniform_cost_search(problem, display=True)` and `astar_search(problem, display=True)` on both word pairs under all three cost measures, making a new DC instance for each search. I used the script from the instructions (same loop and calls, saved as q2_experiments.py). "Nodes expanded" is the first number in the "N paths have been expanded" line printed by the search functions. Reduction = $100 * (\text{UCS expanded} - \text{A* expanded}) / \text{UCS expanded}$.

Word pair	Cost	UCS cost	UCS expanded	A* cost	A* expanded	Reduction
why -> ask	Steps	5	773	5	16	97.93%
why -> ask	Scrabble	9	765	9	13	98.30%
why -> ask	Frequency	38.855	808	38.855	634	21.53%
bear -> duck	Steps	4	691	4	6	99.13%
bear -> duck	Scrabble	11	2981	11	6	99.80%
bear -> duck	Frequency	38.240	1084	38.240	257	76.29%

1. Same optimal cost? Yes. In all six cases UCS and A* returned the same cost: 5, 9, 38.85540652, 4, 11, and 38.2402284. The paths can differ when there are ties, but the costs matched exactly, so there was no discrepancy to investigate. That's what I expected, since UCS is optimal and A* with a consistent heuristic is too.

2. Largest reduction: Scrabble for both pairs (98.30% for why -> ask, 99.80% for bear -> duck). Steps were very close behind (97.93% and 99.13%).
3. Least informative: Frequency for both pairs (21.53% for why -> ask, 76.29% for bear -> duck).
4. Why: It depends on how close h is to the true remaining cost h^* . The closer it is, the fewer nodes A^* has to expand.
 - a) Steps and Scrabble: these heuristics are really close to h^* . For bear -> duck, $h(\text{bear})$ is 4 under steps and 11 under Scrabble, which are exactly the optimal costs, so A^* pretty much goes straight down the optimal path, which is 6 expansions. For why -> ask, $h(\text{why})$ is 3 vs. $h^* = 5$ (steps) and 7 vs. $h^* = 9$ (Scrabble), so it's only off by 2 because of detour letters it can't see. Scrabble has the biggest reduction mostly because UCS does worse there: A lot of letters are only worth 1 point, so there are tons of cheap states for UCS to go through (2981 for bear -> duck vs. 691 under steps).
 - b) Frequency: this heuristic is a lot weaker. It charges every step except the last at $1 + r_{\min}$, but $r_{\min}$ comes from the most common word ("the" = 0.016 or "that" = 0.35), while most words have a rarity around 10-11. Also, the cheapest frequency path often takes more steps than d(s) because it goes through common words, and h only counts d(s) steps. For why -> ask, the optimal path takes 7 steps even though only 3 letters differ, and $h(\text{why}) = 7.27$ vs. $h^* = 38.86$. For bear -> duck, $h(\text{bear}) = 12.28$ vs. $h^* = 38.24$. That's why the reduction is lower, especially for why -> ask. Bear -> duck does better because its optimal path is only 4 steps (the same as d), and the known last step ($1 + \text{rarity}(\text{duck}) = 8.23$) is a bigger part of the total cost.

Q3) if w_1 and w_2 are both dictionary words that differ in exactly one position, then changing w_1 -> w_2 will be legal, and so is w_2 -> w_1 . So any path from W_1 to W_2 , read backwards, is a legal path from W_2 to W_1 through the same words. That is called reversibility, and it only says which moves are allowed. Cost symmetry will be a separate question. Does a move cost the same in both directions? That depends on the cost measure.

1. Yes or no

- a) Steps: Yes, $\text{opt}(W_1 \rightarrow W_2)$ always equals $\text{opt}(W_2 \rightarrow W_1)$.
- b) Scrabble: No, not always.
- c) Frequency: No, not always.

2. Justification

- a) Steps: every action costs 1, so a path costs the same as its reverse (same number of actions). If the best $W_1 \rightarrow W_2$ path has k steps, reversing it gives a $W_2 \rightarrow W_1$ path with k steps, so $\text{opt}(W_2 \rightarrow W_1)$ is less than or equal to $\text{opt}(W_1 \rightarrow W_2)$. The same argument in the other direction gives greater than or equal to, so they're equal. This also means that

if there's no path one way, there's no path the other way. I also checked 150 random 3-letter pairs in both directions with my solver, and they always matched.

- b) Scrabble: the action is reversible, but its cost isn't symmetric. You pay for the letter you put in, and going backwards puts in the old letter instead, which can have a different Scrabble value.
- c) Frequency: the cost is charged for the word you enter, not the word you leave. The starting word is free, but the goal word is paid for. Take any path $W1 = w_0, w_1, \dots, w_k = W2$:

forward cost = sum for $j = 1..k$ of $(1 + \text{rarity}(w_j))$

reverse cost = sum for $j = 0..k-1$ of $(1 + \text{rarity}(w_j))$

forward - reverse = $\text{rarity}(W2) - \text{rarity}(W1)$

This difference is the same for every path, so the best path is the same in both directions, and the two optimal costs differ by exactly $\text{rarity}(W2) - \text{rarity}(W1)$. They're only equal when both words have the same rarity.

3. Counterexamples (for the "no" answers)

- a) Steps: not needed, since the answer is yes.
- b) Scrabble: dog $\rightarrow$ cog costs 3 (insert c), but cog $\rightarrow$ dog costs 2 (insert d). These are both optimal. The words only differ in the first position, so any path dog $\rightarrow$ cog has to put a c there at some point (cost greater than or equal to 3), and the one-step path costs exactly 3. In the same way, cog $\rightarrow$ dog costs greater than or equal to 2 and the one-step path costs 2. My solver agrees: $dc(\text{dog}, \text{cog}, \text{scrabble}) = 3.00$ and $dc(\text{cog}, \text{dog}, \text{scrabble}) = 2.00$.
- c) Frequency: why $\rightarrow$ ask costs 38.855 (why way was has his ais ass ask), but ask $\rightarrow$ why costs 37.755 (the same path reversed). The difference is 1.100, which is exactly $\text{rarity}(\text{ask}) - \text{rarity}(\text{why}) = 4.233 - 3.133$. Another one: dog $\rightarrow$ cat = 23.022 and cat $\rightarrow$ dog = 21.955, a difference of 1.067 = $\text{rarity}(\text{cat}) - \text{rarity}(\text{dog})$.

Q4) 1. Model, prompt, and proposed heuristic

Model: Gemini 2.5 Pro

Exact prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

What it proposed: Gemini's main heuristic multiplies h_0 by the cheapest possible step cost:

$h_1(n) = h_0(n) * (1 + r_{\min})$, where $r_{\min}$ is the smallest rarity in the whole dictionary. Its

argument was that any path needs at least $h_0(n)$ steps, and every step costs at least $1 + r_{\min}$. It also suggested a “refined variant”: $h_{\text{refined}}(n)$ = the cheapest cost over all word paths of exactly $h_0(n)$ steps from n to the goal, set to infinity if no such path exists. It claimed this was “even more informative while preserving admissibility.”

2. Formula / pseudocode

$r_{\min} = \min \text{rarity}(w)$ over all words w in the dictionary $\# = 0.016226$ ("the")

$h_0(n)$ = number of positions where n and goal differ

$h_1(n) = h_0(n) * (1 + r_{\min})$

Refined variant (as Gemini described it):

$k = h_0(n)$

$h_{\text{refined}}(n) = \min \text{cost of any path } n \rightarrow \text{goal with exactly } k \text{ steps}$

(infinity if no k -step path exists)

3. Is it admissible?

h_1 : yes. Any path to the goal needs at least $h_0(n)$ steps, since each action fixes at most one letter, and every step costs at least $1 + r_{\min}$. So $h^*(n)$ is greater than or equal to $h_0(n) * (1 + r_{\min}) = h_1(n)$. Since $r_{\min} = 0.016$ greater than 0, h_1 greater than h_0 for every non-goal state, so it's also more informative.

Refined variant: no. It assumes the best path has exactly $h_0(n)$ steps, but under frequency the cheapest path is often longer.

- a) dog $\rightarrow$ cat: the cheapest 3-step path (dog dot cot cat) costs 29.10, but the true optimal is 23.02 (dog don dan can cat, 4 steps). So $h_{\text{refined}} = 29.10$ greater than $h^* = 23.02$.
- b) why $\rightarrow$ ask: there's no 3-step path, so $h_{\text{refined}} = \text{infinity}$, but $h^* = 38.86$.

4. Repair

h_1 doesn't need a fix. The refined variant can't be fixed by just removing the infinity case, since dog $\rightarrow$ cat still overestimates. So I dropped it and used h_1 .

5. Test against h_0

Problem (frequency)	Heuristic	Solution cost	A* expanded	vs. h_0
why $\rightarrow$ ask	h_0	38.855	755	-
why $\rightarrow$ ask	h_1	38.855	754	0.13% fewer
bear $\rightarrow$ duck	h_0	38.240	720	-

bear -> duck	h1	38.240	714	0.83% fewer
--------------	----	--------	-----	-------------

Conclusion: Both found the same optimal costs, but h1 barely helped (1 and 6 fewer nodes) because $r_{\min} = 0.016$ makes h1 basically $h_0 * 1.016$. So h1 is safe, but the evidence doesn't really support using it over h0. A better option is to use the fact that the last step always costs 1 + rarity(goal), like my Q1 heuristic, which expanded only 634 and 257 nodes.

Sources: course slides, the p8.py example, and the supplied starter code. Q2 uses the script given in the instructions.

AI use: I used Claude (Anthropic) to help understand the questions, and Q4 used Gemini 2.5 Pro as required.